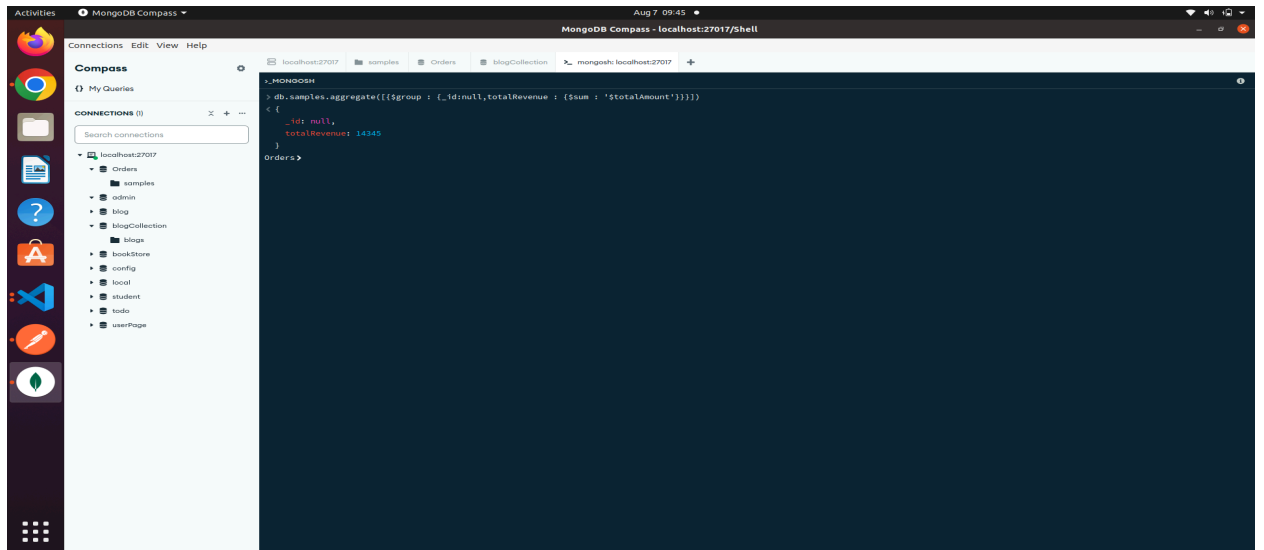


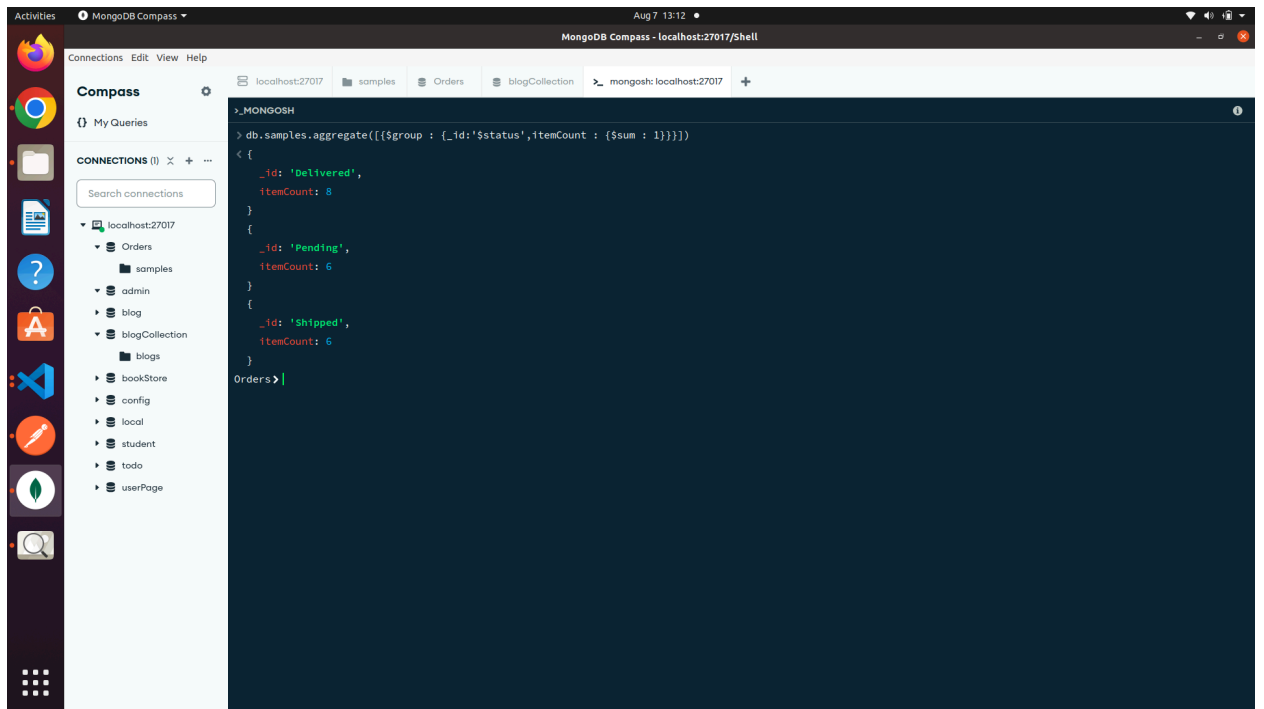
MongoDb Assessment

Aggregation :

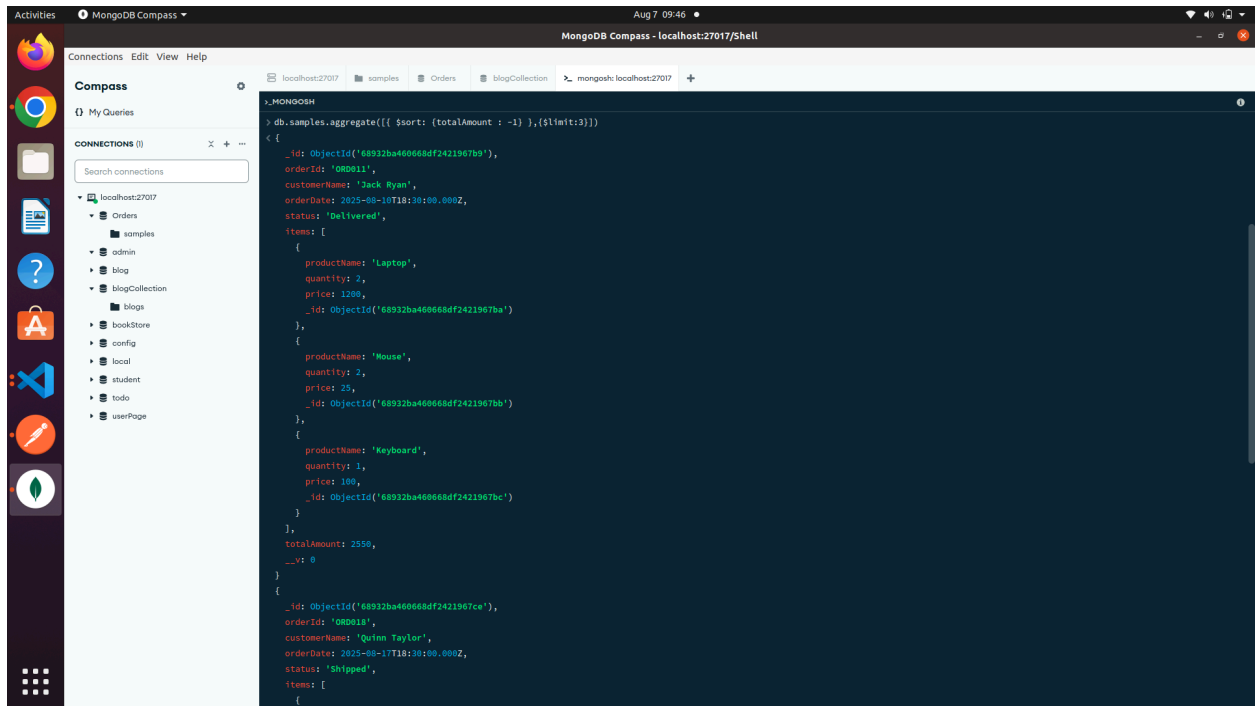
1. Find total revenue generated (sum of totalAmount).



2. Find total number of orders by status (Pending, Shipped, Deliver)



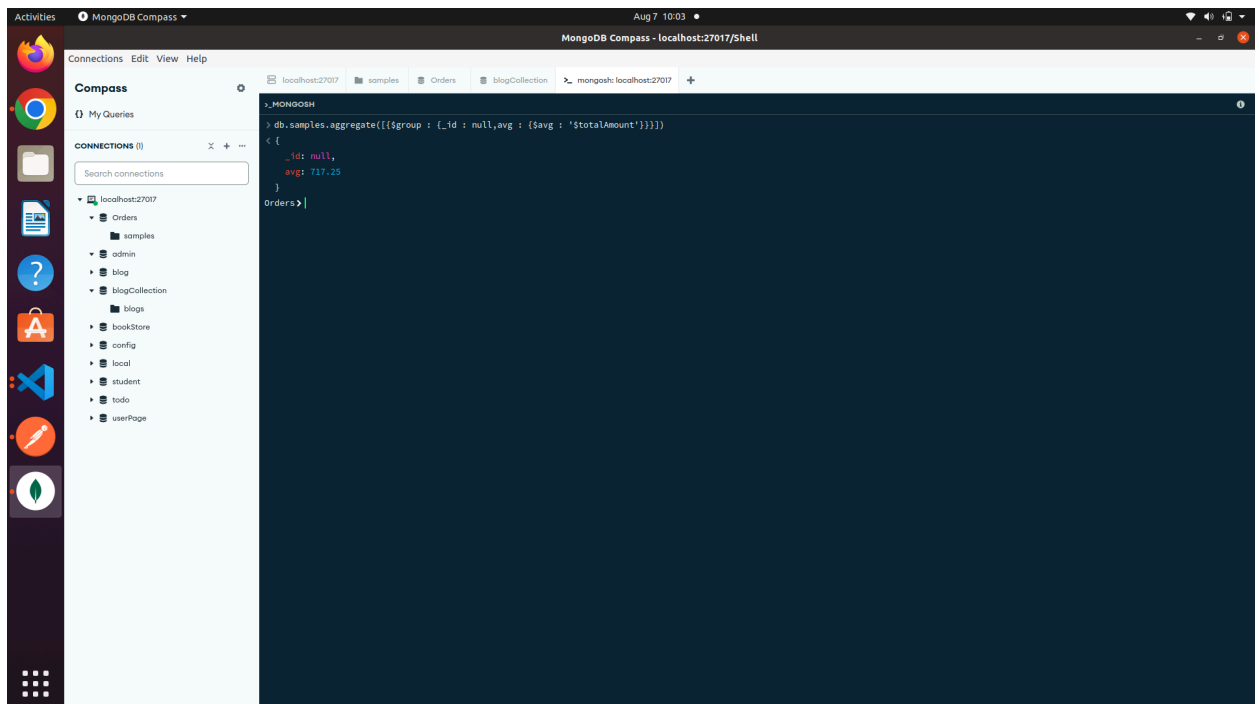
3. Find the top 3 customers who spent the most (sort by totalAmount).



The screenshot shows the MongoDB Compass interface. The left sidebar displays the database structure with collections: samples, Orders, blogCollection, blogs, bookstore, config, local, student, todo, and userPage. The main panel shows a MongoDB Shell with the following query and result:

```
> db.samples.aggregate([{$sort: {totalAmount: -1}},{$limit:3}])
< [
  {
    _id: ObjectId('68932ba460668df2421967b9'),
    orderId: 'ORD001',
    customerName: 'Jack Ryan',
    orderDate: 2025-08-10T18:10:00.000Z,
    status: 'Delivered',
    items: [
      {
        productName: 'Laptop',
        quantity: 2,
        price: 1200,
        _id: ObjectId('68932ba460668df2421967ba')
      },
      {
        productName: 'Mouse',
        quantity: 2,
        price: 25,
        _id: ObjectId('68932ba460668df2421967bb')
      },
      {
        productName: 'Keyboard',
        quantity: 1,
        price: 100,
        _id: ObjectId('68932ba460668df2421967bc')
      }
    ],
    totalAmount: 2550,
    __v: 0
  }
]
```

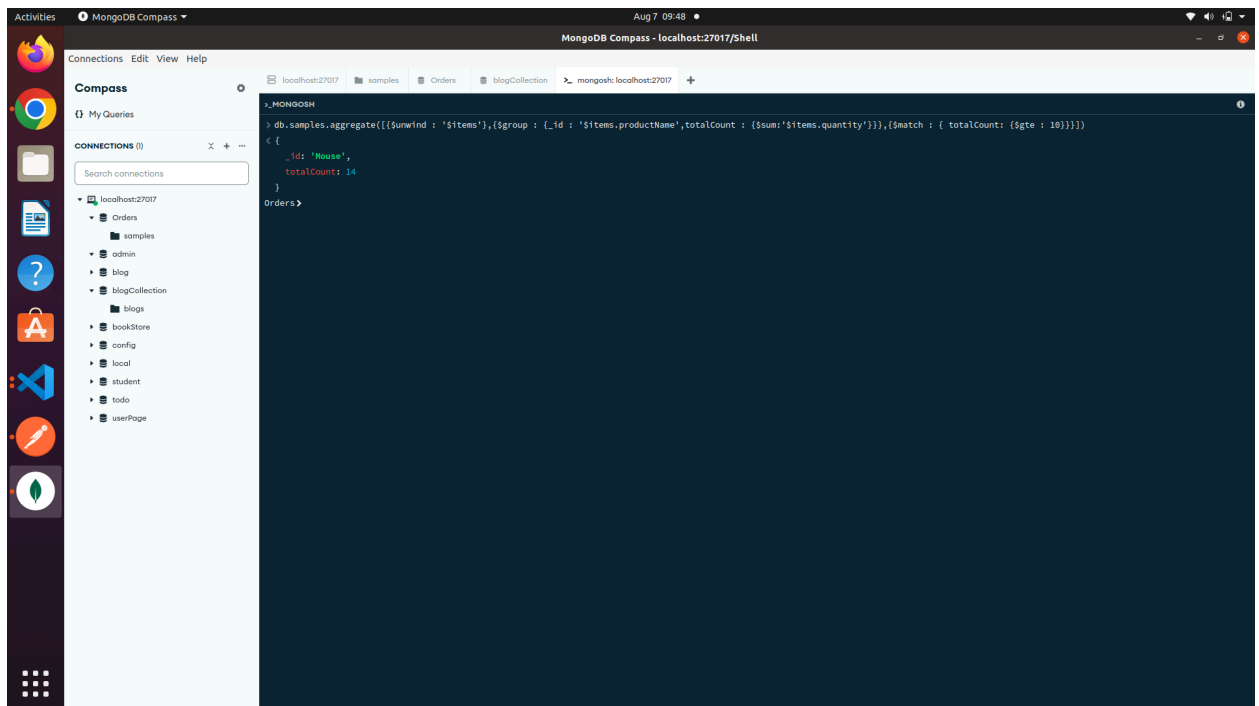
4. Get the average order amount per customer.



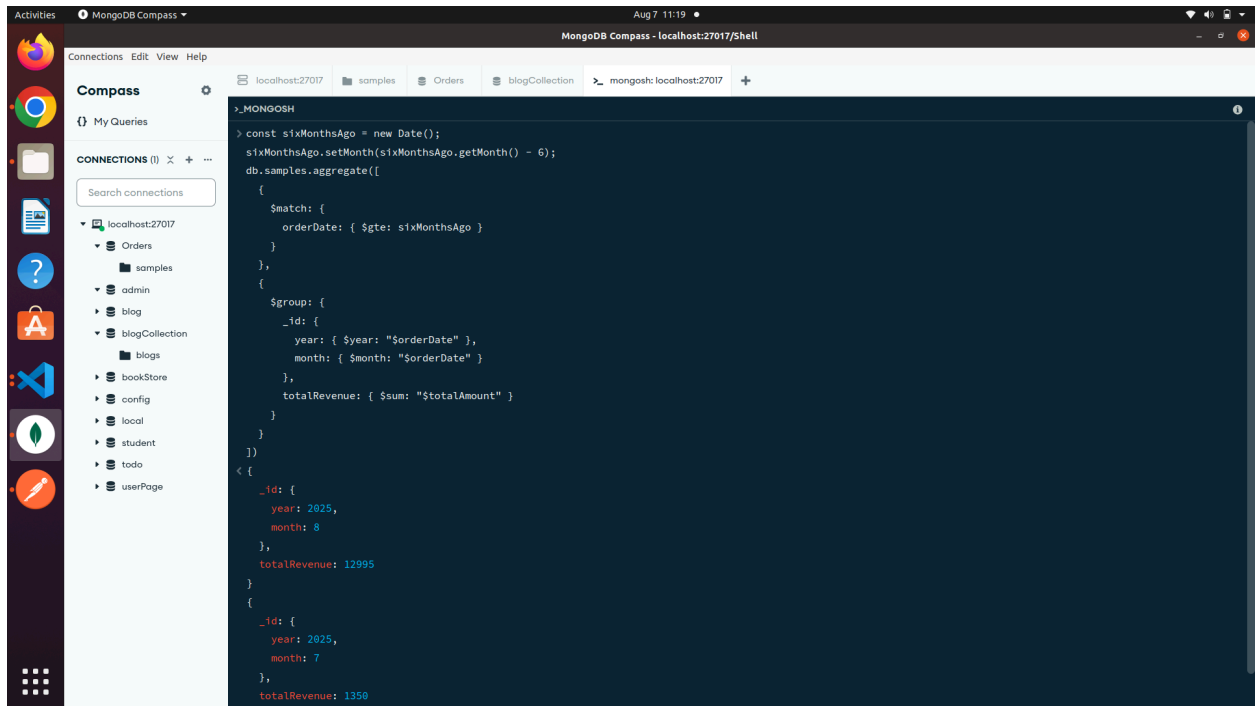
The screenshot shows the MongoDB Compass interface. The left sidebar displays the database structure with collections: samples, Orders, blogCollection, blogs, bookstore, config, local, student, todo, and userPage. The main panel shows a MongoDB Shell with the following query and result:

```
> db.samples.aggregate([{$group: {_id: null, avg: {$avg: '$totalAmount'}}}])
< [
  {
    _id: null,
    avg: 717.25
  }
]
```

5.Find products that were sold more than 10 times (total quantity).



6. List monthly revenue (group by month-year) for the last 6 months.



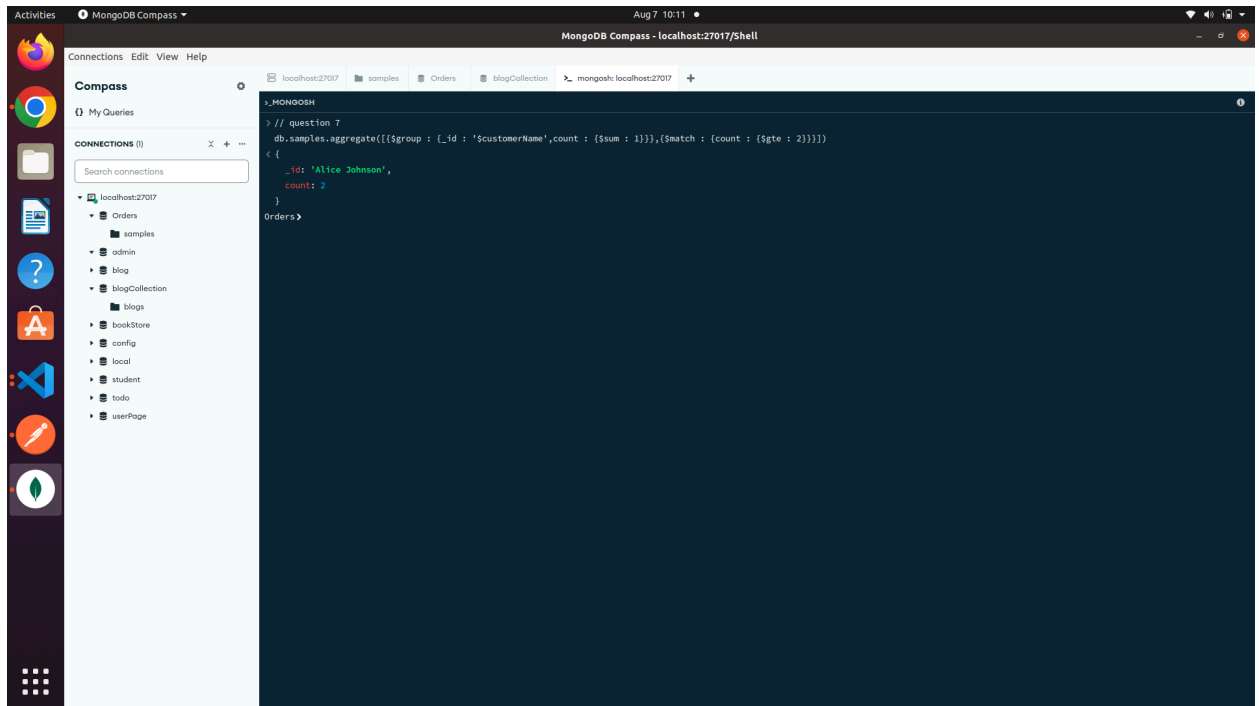
The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' panel with a search bar and a list of databases. The 'localhost:27017' database is selected, showing collections: Orders, samples, admin, blog, blogCollection, blogs, bookStore, config, local, student, todo, and userPage. The main panel shows the 'MongoDB Compass - localhost:27017/Shell' window with a MongoDB shell prompt. The following aggregation query is entered:

```
> const sixMonthsAgo = new Date();
sixMonthsAgo.setMonth(sixMonthsAgo.getMonth() - 6);
db.samples.aggregate([
  {
    $match: {
      orderDate: { $gte: sixMonthsAgo }
    }
  },
  {
    $group: {
      _id: {
        year: { $year: "$orderDate" },
        month: { $month: "$orderDate" }
      },
      totalRevenue: { $sum: "$totalAmount" }
    }
  }
])
```

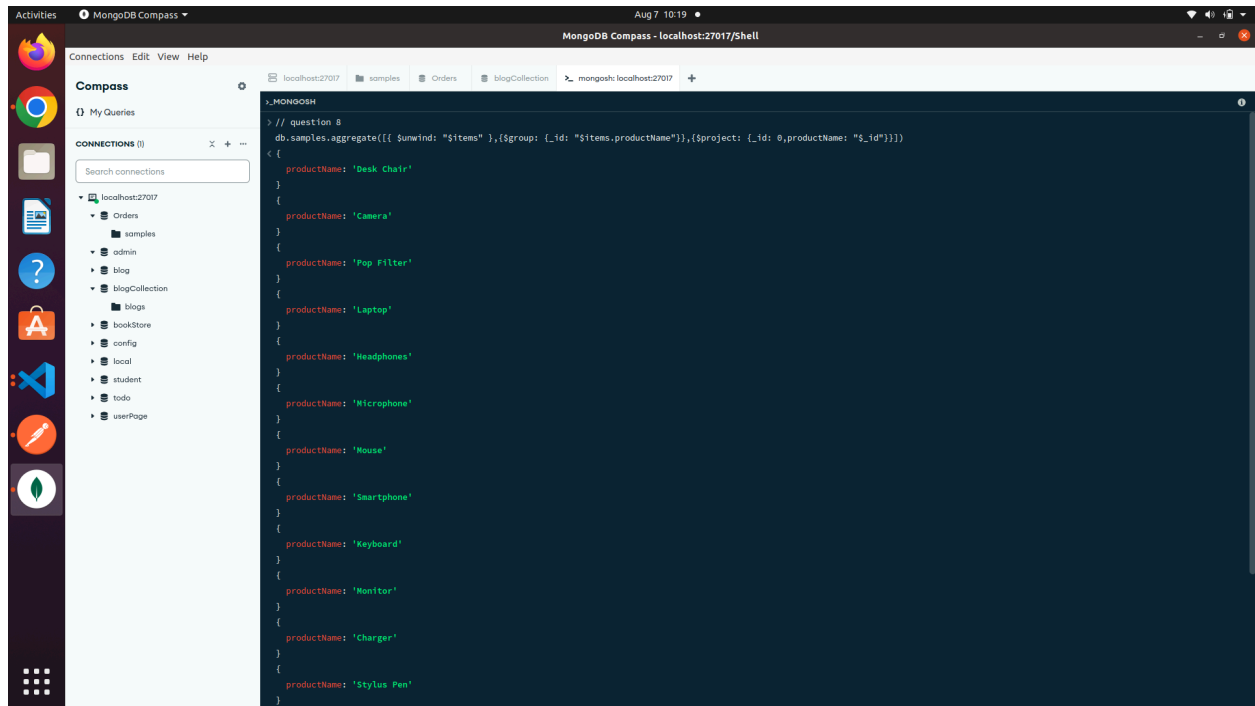
The results of the aggregation are displayed below the query:

```
< {
  "_id": {
    "year": 2025,
    "month": 8
  },
  "totalRevenue": 12995
},
{
  "_id": {
    "year": 2025,
    "month": 7
  },
  "totalRevenue": 1350
}
```

7. Find all customers who placed more than 2 orders.



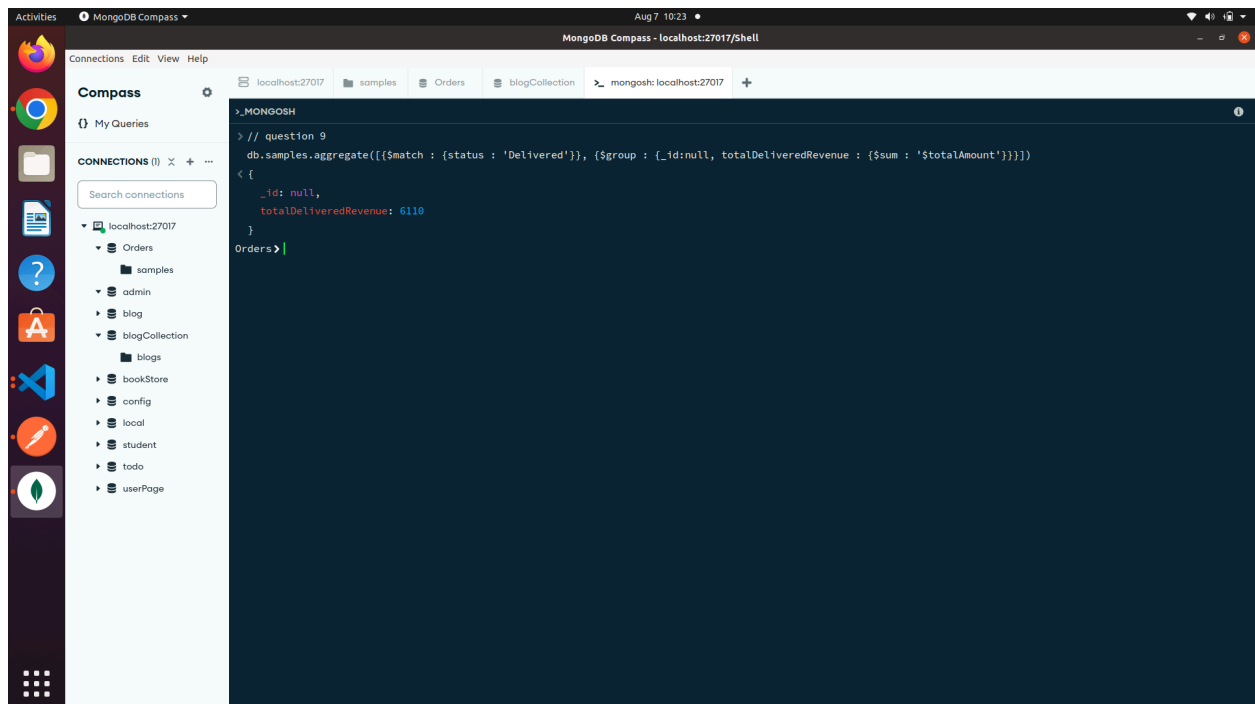
8. Extract only the product names from all orders using \$unwind and \$project.



The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' panel with a tree view of databases and collections. The main editor area shows a query in the 'MONGOSH' shell. The query uses the \$unwind operator to deconstruct the 'items' array and the \$project operator to select only the 'productName' field. The result is a list of product names.

```
> // question 8
db.samples.aggregate([{$unwind: "$items"},{$group: {_id: "$items.productName"}},{$project: {_id: 0,productName: "$_id"}}])
< {
  productName: 'Desk Chair'
}
{
  productName: 'Camera'
}
{
  productName: 'Pop Filter'
}
{
  productName: 'Laptop'
}
{
  productName: 'Headphones'
}
{
  productName: 'Microphone'
}
{
  productName: 'Mouse'
}
{
  productName: 'Smartphone'
}
{
  productName: 'Keyboard'
}
{
  productName: 'Monitor'
}
{
  productName: 'Charger'
}
{
  productName: 'Stylus Pen'
}
```

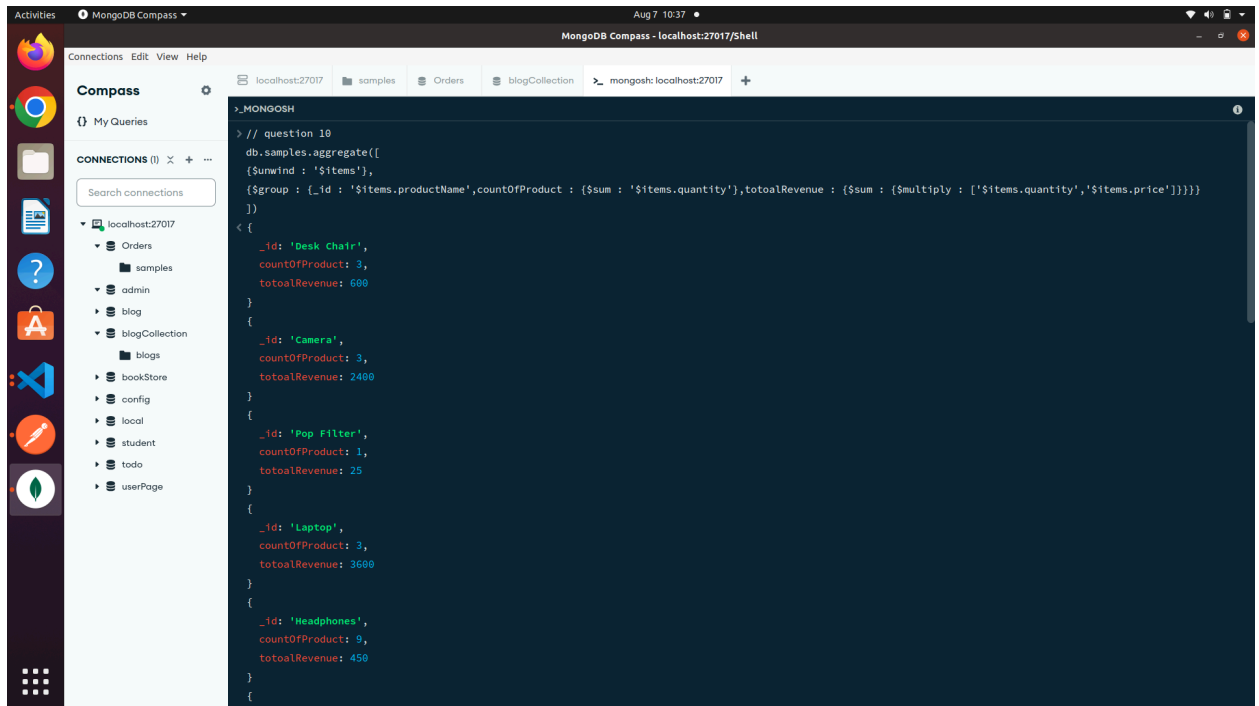
9. Apply filtering using \$match (only Delivered orders) and then calculate revenue.



The screenshot shows the MongoDB Compass interface. The left sidebar displays the 'Connections' panel with a tree view of databases and collections. The main editor area shows a query in the 'MONGOSH' shell. The query uses the \$match operator to filter for 'Delivered' orders and the \$group operator to calculate the total revenue. The result shows a single document with the total delivered revenue of 6110.

```
> // question 9
db.samples.aggregate([{$match: {status: 'Delivered'}},{$group: {_id:null, totalDeliveredRevenue: {$sum: '$totalAmount'}}})
< {
  _id: null,
  totalDeliveredRevenue: 6110
}
Orders>
```

10. Calculate total quantity and total revenue per product (use \$unwind and \$group).



The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel lists the 'localhost:27017' connection, expanded to show databases like 'Orders', 'samples', 'admin', 'blog', 'blogCollection', 'blogs', 'bookStore', 'config', 'local', 'student', 'todo', and 'userPage'. The main panel displays a MongoDB Shell session with the following aggregation query:

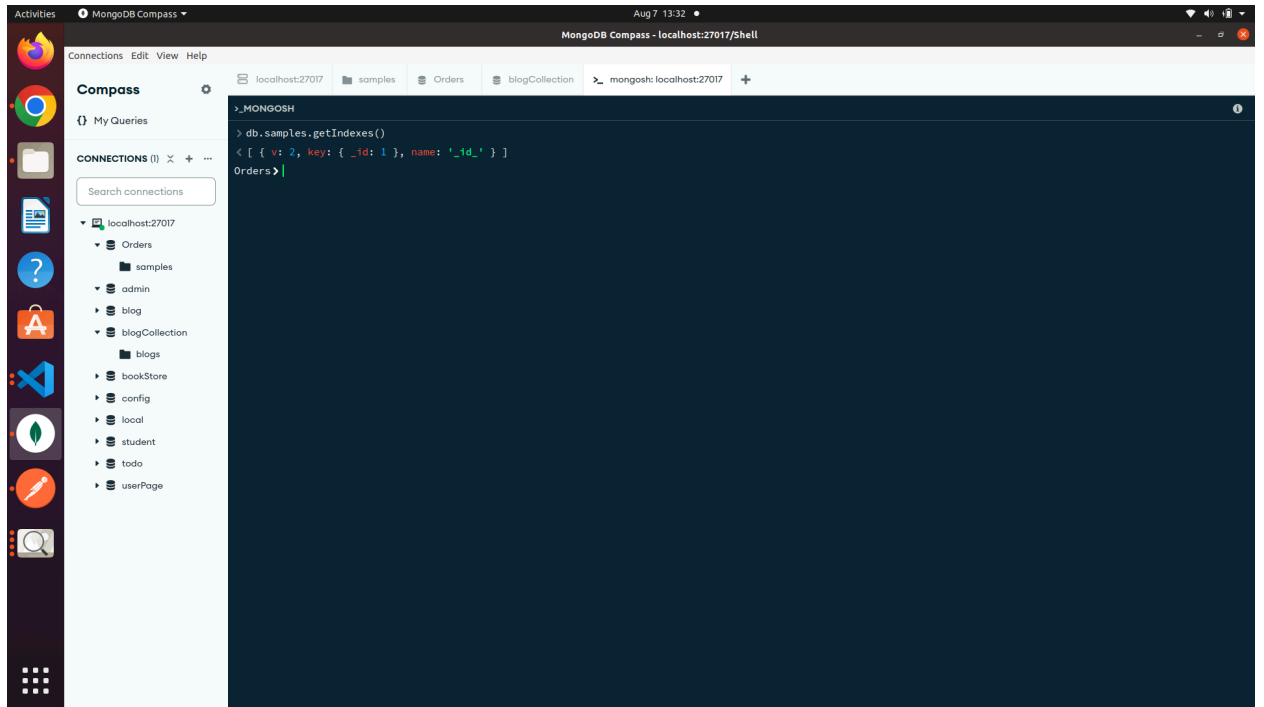
```
> // question 10
db.samples.aggregate([
  {$unwind : '$items'},
  {$group : {_id : '$items.productName',countOfProduct : {$sum : '$items.quantity'},totalRevenue : {$sum : {$multiply : ['$items.quantity','$items.price']}}}}
])
```

The results of the aggregation are shown in a JSON array:

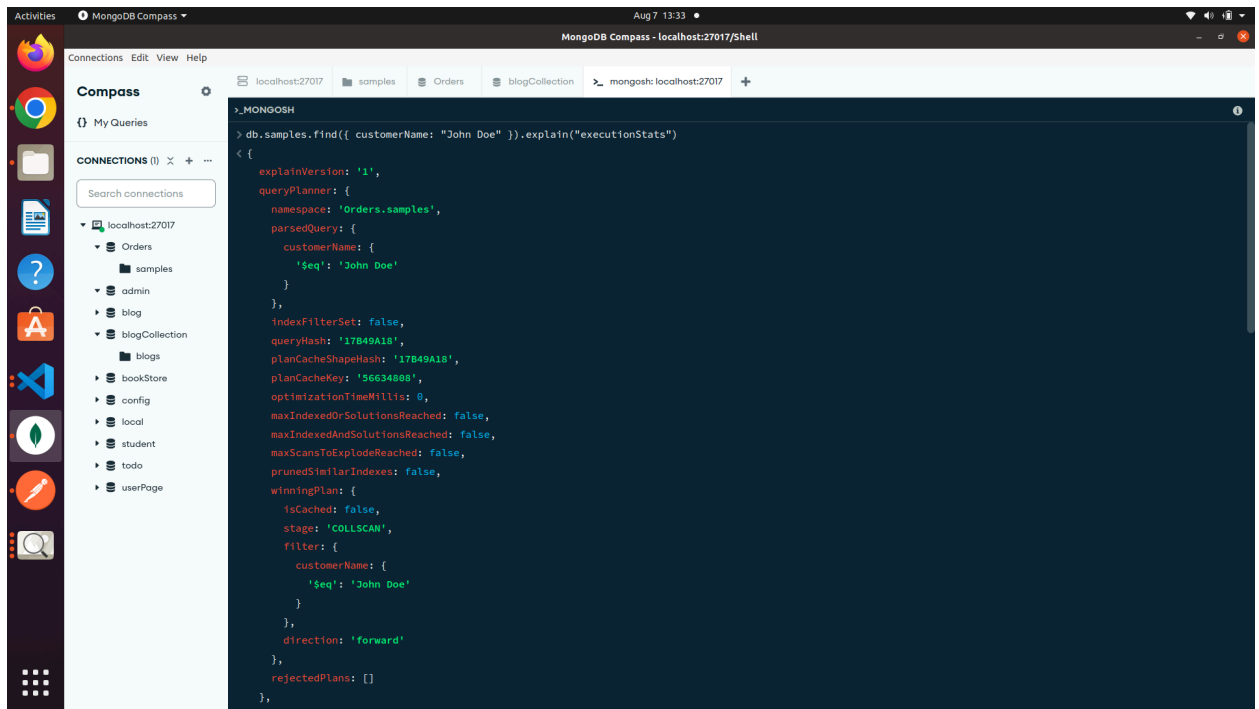
```
< [
  {
    "_id": "Desk Chair",
    "countOfProduct": 3,
    "totalRevenue": 600
  },
  {
    "_id": "Camera",
    "countOfProduct": 3,
    "totalRevenue": 2400
  },
  {
    "_id": "Pop Filter",
    "countOfProduct": 1,
    "totalRevenue": 25
  },
  {
    "_id": "Laptop",
    "countOfProduct": 3,
    "totalRevenue": 3600
  },
  {
    "_id": "Headphones",
    "countOfProduct": 9,
    "totalRevenue": 450
  }
]
```


Indexing :

1. Check indexes on the collection.

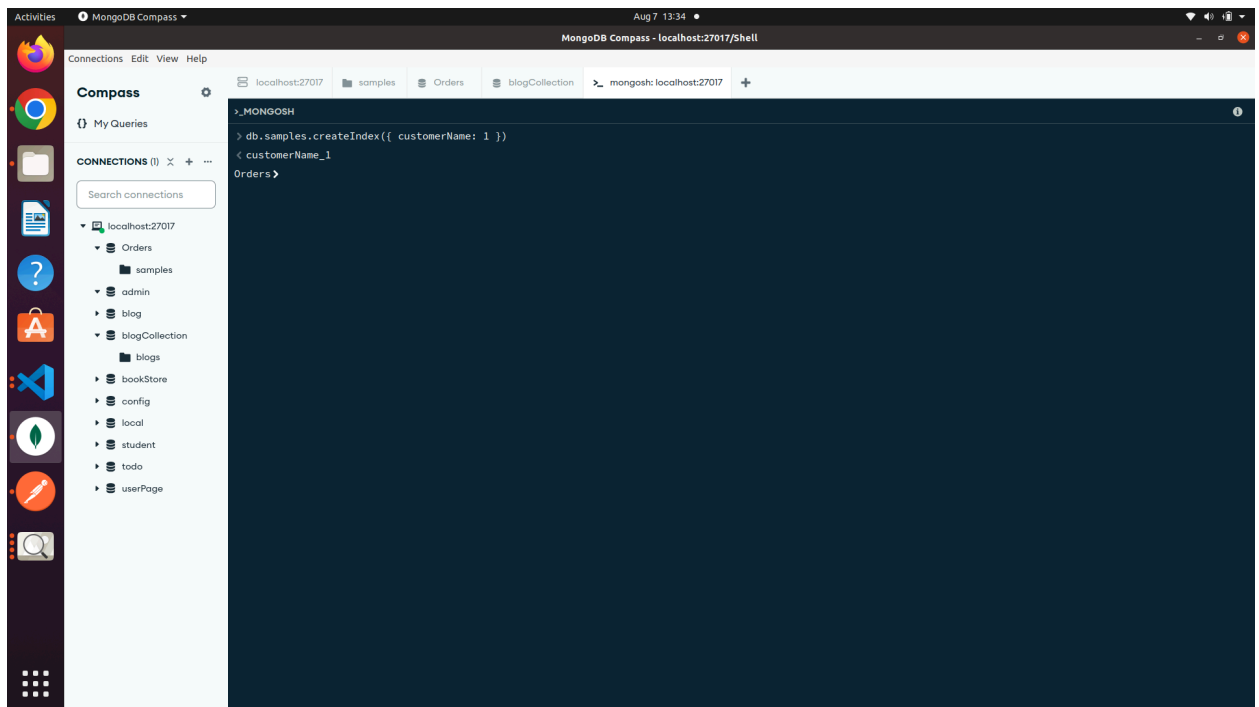


2.Create an index on customerName. Run a query filtering by customerName and check performance using explain("executionStats").



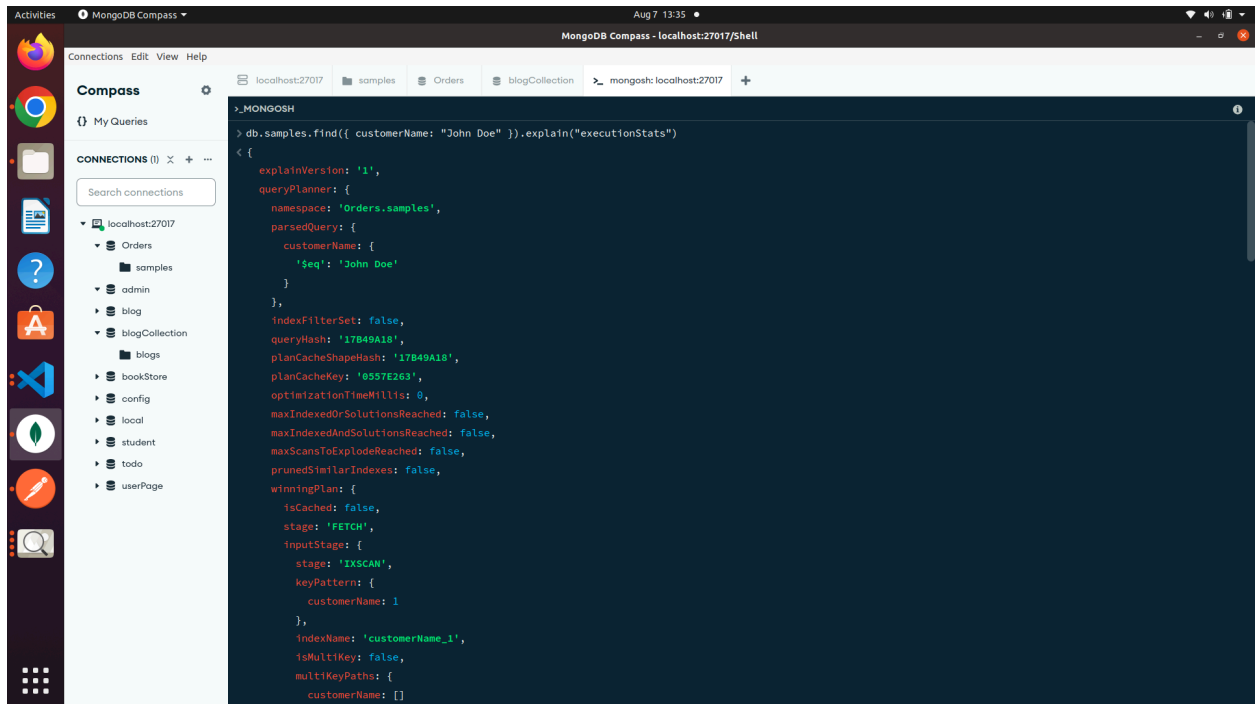
The screenshot shows the MongoDB Compass interface. The left sidebar displays the database structure with collections: Orders, samples, admin, blog, and blogCollection. The main panel shows a MongoDB shell session with the following command and output:

```
> db.samples.find({ customerName: "John Doe" }).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Orders.samples',
    parsedQuery: {
      customerName: {
        '$eq': 'John Doe'
      }
    },
    indexFilterSet: false,
    queryHash: '17849A18',
    planCacheShapeHash: '17849A18',
    planCacheKey: '56634888',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExploreReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        customerName: {
          '$eq': 'John Doe'
        }
      },
      direction: 'forward'
    },
    rejectedPlans: []
  },
```

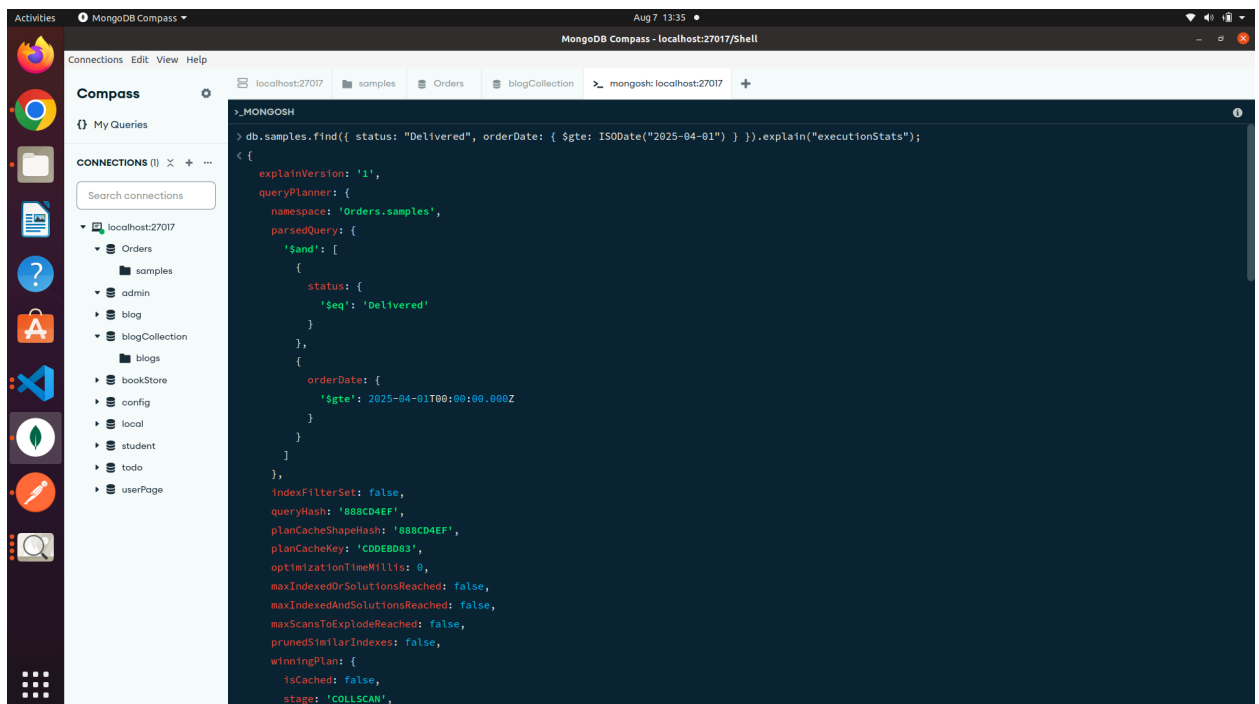


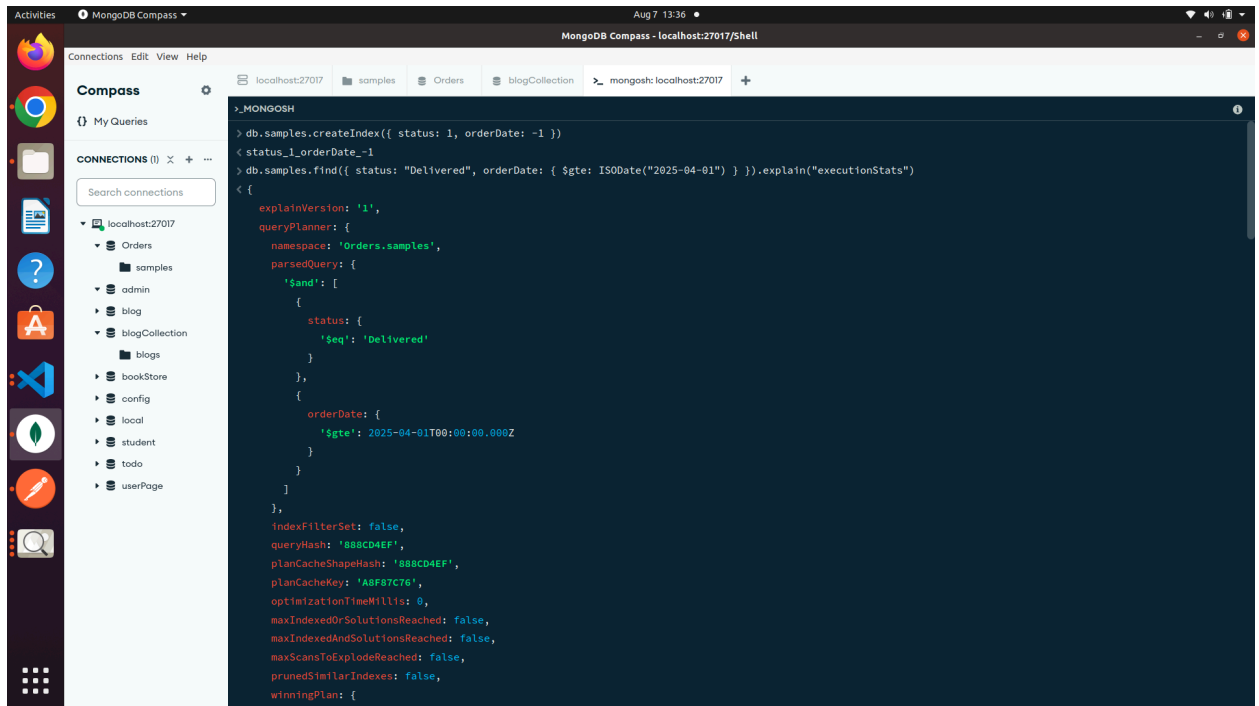
The screenshot shows the MongoDB Compass interface after creating an index. The left sidebar is the same. The main panel shows a MongoDB shell session with the following command and output:

```
> db.samples.createIndex({ customerName: 1 })
< customerName_1
Orders>
```

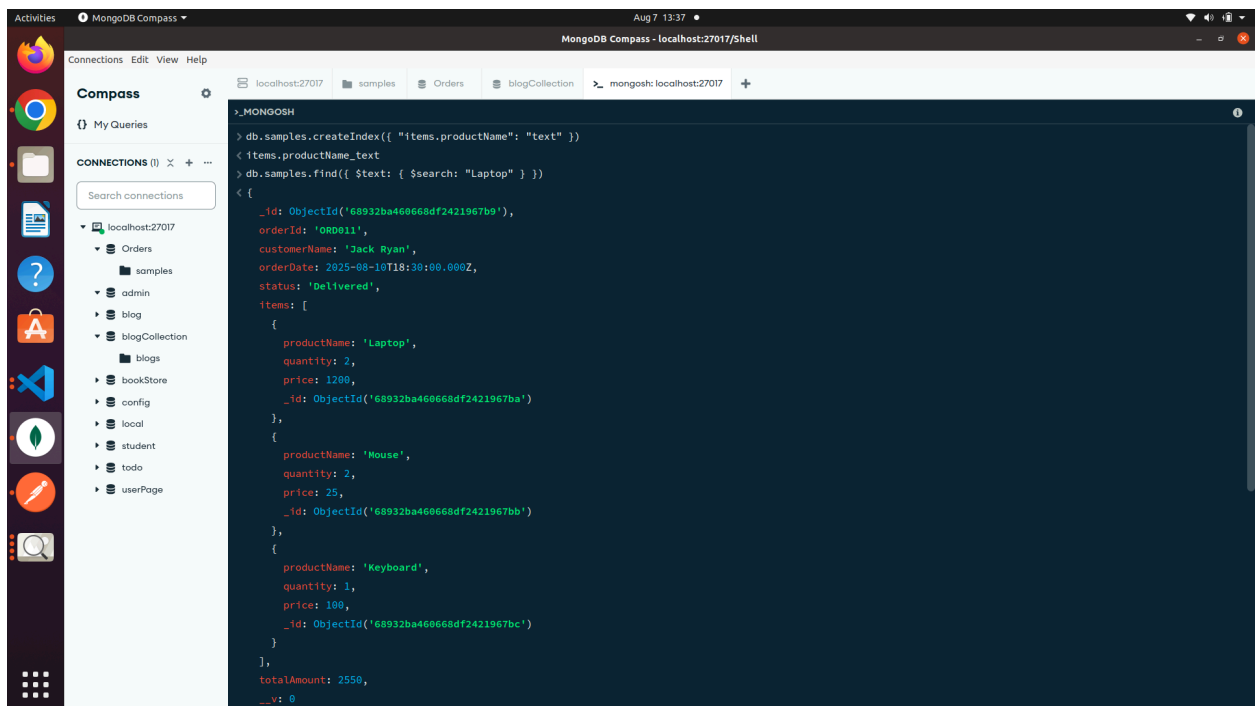


3. Create a compound index on status and orderDate. Run a query filtering by both and compare performance (before vs after).

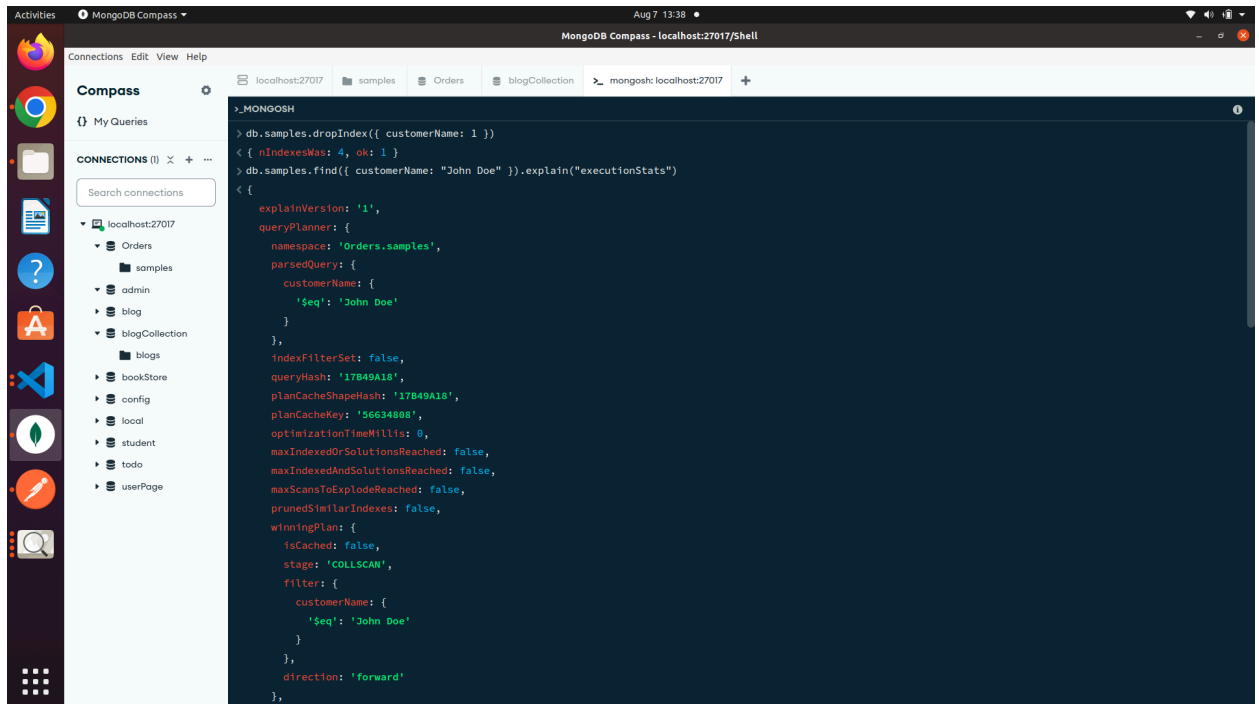




4. Create a text index on items.productName. Perform a text search for a product.



5.Drop an index and observe performance difference.



The screenshot shows the MongoDB Compass application interface. On the left, the 'Connections' panel lists the local connection 'localhost:27017' with a tree view of databases including 'Orders', 'samples', 'admin', 'blog', 'blogCollection', 'blogs', 'bookStore', 'config', 'local', 'student', 'todo', and 'userPage'. The main panel is titled 'MongoDB Compass - localhost:27017/Shell' and contains a MongoDB shell session. The session shows the execution of the command `db.samples.dropIndex({ customerName: 1 })`, which returns `< { nIndexesWas: 4, ok: 1 }`. This is followed by `db.samples.find({ customerName: "John Doe" }).explain("executionStats")`, which returns a detailed execution plan. The plan indicates a 'COLLSCAN' stage, meaning the database is scanning all documents in the collection because the index has been dropped. The plan also shows the query filter for `customerName: { '$eq': 'John Doe' }` and the direction 'forward'.

```
>_MONGOSH
> db.samples.dropIndex({ customerName: 1 })
< { nIndexesWas: 4, ok: 1 }
> db.samples.find({ customerName: "John Doe" }).explain("executionStats")
< {
  explainVersion: '1',
  queryPlanner: {
    namespace: 'Orders.samples',
    parsedQuery: {
      customerName: {
        '$eq': 'John Doe'
      }
    },
    indexFilterSet: false,
    queryHash: '17B49A18',
    planCacheShapeHash: '17B49A18',
    planCacheKey: '56634888',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        customerName: {
          '$eq': 'John Doe'
        }
      },
      direction: 'forward'
    },
  },
}
```

Observations And Learning :

1. Indexes improves query performance especially when querying large collections.
2. `.explain('executeStats')` is essential for understanding how MongoDB runs your queries.
3. Performance of query is measured through the number of documents examined i.e `totalDocsExamined`: 8
4. Compound indexes help multi-field filters perform better.
5. Performance of queries which involves compound indexes depends on the order of fields in index.

Sample JSON :

```
{
  orderId: "ORD001",
  customerName: "Alice Johnson",
  orderDate: new Date("2025-08-01"),
  status: "Delivered",
  items: [
    { productName: "Laptop", quantity: 1, price: 1200 },
    { productName: "Mouse", quantity: 2, price: 25 },
    { productName: "Keyboard", quantity: 1, price: 100 }
  ],
  totalAmount: 1350
}
```

Aggregation :

Aggregation in MongoDB is a way to process and transform data within a collection using a pipeline of stages. Each stage performs an operation on the data (e.g., filtering, grouping, sorting), and passes the results to the next stage similar to a data processing pipeline.

Benefits :

1. Performs server-side computations avoids pulling data into your app.
2. Supports filtering, projecting, sorting, joining, and more.
3. The heavy lifting happens in the database, not in your app code.
4. Designed to handle large volumes of data efficiently.